Preserve, Do Not Reify: Controlled Evidence for Keeping N-ary Facts Whole in Knowledge Graph Completion

[Author names]
[Affiliation, Department, City, Country]
[Corresponding author e-mail]

Manuscript draft, 4 November 2026

Abstract

Knowledge graphs store many facts with more than two participants, and the standard way to write them down—RDF n-ary patterns, Wikidata qualifiers—*reifies* each fact into an intermediate node joined to its participants by binary edges. Hyper-relational completion models instead keep the fact whole, but prior comparisons either changed the model together with the representation or, when they held the model fixed, did not use the standard reification as the control. We hold the model fixed and change only the representation. Across three studies, nine pre-registered judged comparisons and one condition pre-specified as report-only all favoured the representation that keeps the fact whole, on all five seeds. Study 1 uses one role-aware hypergraph encoder fed either hyperedges or the reified graph, under conditions that include an inductive protocol, a matched output head, a frozen sentence encoder, real Wikidata facts and a reified control with twice the depth. Study 2 leaves the public StarE code unmodified, runs its standard protocol on WD50K(100) and swaps only the graph its encoder sees; the native graph reaches test MRR 0.6553, reproducing the published 0.654, against 0.6485 for the reified graph. Study 3 places our encoder inside the same pipeline: preserving reaches 0.6648, reifying 0.6561 and removing qualifiers from the encoder 0.6519. At the standard budget the gaps are small (0.007–0.013 MRR) but never reverse. With StarE's encoder, however, the reified graph learns faster—a 76-epoch pilot had it ahead by 0.098 MRR—so short-budget comparisons can report the opposite order. We claim neither state-of-the-art accuracy—HAHE's public code, run on the identical pipeline, beats our encoder on all five seeds (0.6784 vs. 0.6648)—nor any retrieval benefit; under a frozen sentence encoder, a structure-free text baseline retrieves facts better than every structured variant.

Keywords: N-ary knowledge graph completion; hyper-relational knowledge graph; reification; hypergraph neural network; link prediction; controlled comparison

1 Introduction

Many facts in a knowledge base have more than two participants. "Kim led the research lab as team lead from 2021 to 2023" is one fact about a person, an organisation, a role and a time span, not four separate facts. The data models that store such facts offer binary edges: an RDF property is a binary relation (Cyganiak et al., 2014), a W3C Working Group Note describes writing n-ary relations through an intermediate individual (Noy and Rector, 2006), and Wikidata attaches qualifiers to a main statement (Vrandečić and Krötzsch, 2014). In each

case one fact is *reified*: a node stands for the fact and binary edges connect it to each participant. This practice reflects the binary data model, not evidence about what a learned model needs.

Link prediction on n-ary facts has moved the other way. Embedding models score whole n-ary tuples (Wen et al., 2016; Guan et al., 2019; Fatemi et al., 2020), convolutional and attention models read a main triple together with its qualifiers (Rosso et al., 2020; Wang et al., 2021; Hu et al., 2023), and message-passing models propagate information across facts while keeping qualifiers attached (Galkin et al., 2020; Luo et al., 2023; Lee and Whang, 2025; Lee et al., 2026; Ji et al., 2026). These models rest on the premise that keeping a fact whole is better than splitting it. The direct evidence for the premise is thinner than the volume of work suggests. When HINGE was compared with reified inputs, the reified inputs went to different (binary) models (Rosso et al., 2020); when m-TransH compared decompositions, adding a fact node to the n-ary data *improved* accuracy (Wen et al., 2016); and a recent analysis found that a qualifier-free path-based GNN matches or exceeds specialised hyper-relational models on WD50K (Wang et al., 2025). None of these holds the model fixed and swaps a preserved fact for its standard reification.

We do exactly that. Our question is narrow: *with the encoder, its training budget and the information it receives held fixed, and its parameter count fixed or within a few percent, does it matter whether an n-ary fact enters whole or as its standard reification?* We answer it three times, with increasingly standard machinery.

- **Study 1** uses one role-aware hypergraph encoder and changes only its input graph. Six conditions were judged—the baseline, an inductive protocol, a matched output head, a frozen multilingual sentence encoder, real Wikidata facts (WD50K(100)), and a reified control with twice the layers and 14% more parameters—and a seventh, the three synthetic controls combined, was pre-specified as report-only.

- **Study 2** leaves the public StarE code (Galkin et al., 2020) unmodified, runs its standard protocol and budget, and swaps the encoder graph for the reified one through a one-method subclass that runs StarE's own qualifier-free convolution on it.

- **Study 3** places the encoder of Study 1 inside the same standard pipeline and adds a third arm that removes the qualifiers from the encoder graph, addressing the qualifier-free result of Wang et al. (2025).

To place the encoder among published models, we also run HAHE's public code (Luo et al., 2023) on the identical pipeline and judge it against our encoder by the same rule. That comparison is between models, not representations, and it is not one of the nine.

Our contributions are as follows.

1. A controlled comparison in which the representation is the only intended difference. Nine two-arm comparisons were judged by a rule fixed before the runs, and all nine—together with the report-only combined condition—favoured the unreified representation (a hyperedge in Studies 1 and 3, StarE's qualifier-carrying edge in Study 2) on 5 of 5 seeds.

2. Replication outside our own code and protocol. The direction holds on an external backbone whose published number we reproduce (0.6553 vs. 0.654), and for our encoder under the same standard protocol.

3. A budget effect that can flip conclusions. With StarE's encoder the reified graph learns faster early in training: a 76-epoch pilot had it ahead by 0.098 MRR, and in the full runs it leads at most evaluation points before epoch 120. Comparisons run on short budgets can therefore report the opposite of the converged order.

4. An honest position in the literature. On WD50K(100) our encoder sits at the level of StarE, below HAHE when we run HAHE's public code on the identical pipeline (5 of 5 seeds), and far below GLoRE; we report where a structure-free text encoder beats every structured variant, and where an attempt to carry the advantage into a language model failed.

2 Related work

N-ary and hyper-relational completion. Three formalisms coexist. Role–value formulations treat a fact as a set of role:value pairs (Guan et al., 2019, 2021); knowledge hypergraphs treat it as a relation over an ordered tuple (Wen et al., 2016; Fatemi et al., 2020); hyper-relational graphs keep a main triple and attach qualifier pairs (Rosso et al., 2020; Galkin et al., 2020). Within the last family, models differ in how a qualifier enters. StarE folds qualifier pairs into the feature of the main edge of a CompGCN-style encoder (Vashishth et al., 2020; Galkin et al., 2020), so a qualifier value contributes to messages but receives none from its own statement. GRAN treats each fact as a small heterogeneous graph processed by edge-biased attention (Wang et al., 2021), and HyperFormer combines a token sequence with one-hop neighbour aggregation (Hu et al., 2023). HAHE adds a global hypergraph in which qualifier values are members of the fact (Luo et al., 2023); MAYPL passes messages over structure induced by qualifiers (Lee and Whang, 2025); KREPE sums relation–entity pairs into a fact representation and passes context messages across facts (Lee et al., 2026); and GLoRE encodes relation–entity pairs jointly at a global and a local level (Ji et al., 2026). HEHRGNN treats each fact as a hyperedge hub (Rajagopalamenon and Cheramangalath, 2026), and hyper-relational structure has been used in applications such as next-POI recommendation (Zhang et al., 2023). The encoder we study belongs to the participant family: every participant of a fact sends and receives messages through the fact.

Evidence about the representation itself. Wen et al. (2016) compared the same model family on JF17K converted three ways: a pairwise (star-to-clique) expansion reached Hits@10 49.26%, direct n-ary instances 68.73%, and n-ary instances with an added fact-ID role 80.82%. The last result points in the direction opposite to ours, but the fact node there is *added* to an intact n-ary fact, whereas reification *replaces* the fact by the node and its binary edges. Rosso et al. (2020) gave eight binary embedding models a reified version of each fact; reification did not help them, and HINGE, a different model, beat all of them. Fatemi et al. (2020) showed that existing embedding methods do not work well out of the box on graphs binarised by reification. Galkin et al. (2020) showed that removing qualifiers from StarE costs a large margin on WD50K(100) (0.654 vs. 0.380 MRR), and Wang et al. (2025) found that NBFNet (Zhu et al., 2021) without qualifiers (0.360 on WD50K) matches or exceeds StarE as re-run by those authors (0.346; originally 0.349), and that adding qualifiers as extra triples makes NBFNet worse. The GLoRE authors report that node-centric global aggregation confuses roles and loses 0.13 MRR on WD50K against pair-based aggregation (0.348 vs. 0.478) (Ji et al., 2026). Each

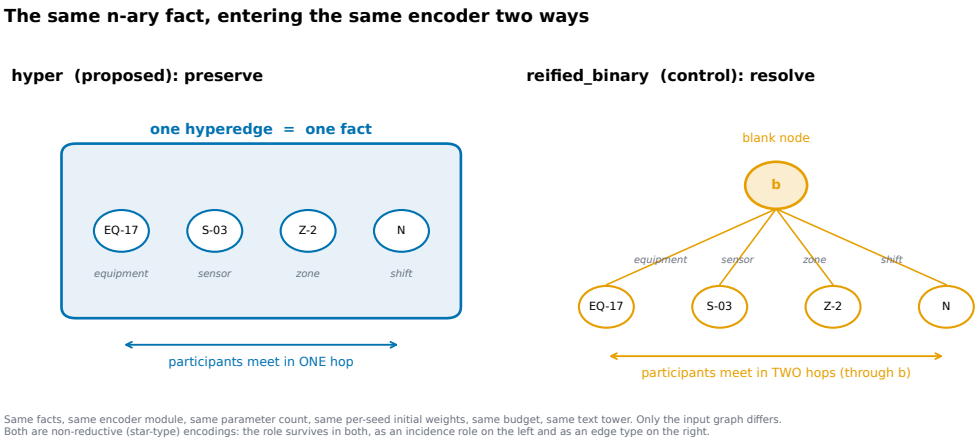


Figure 1: The same n-ary fact entering the same encoder in two ways. Left (preserve): one hyperedge, members meet in one hop. Right (reify): a statement node with one binary edge per member, members two hops apart. Roles survive on both sides—as incidence roles on the left, as edge types on the right. What differs is the path, not the information.

of these varies something besides the representation, or varies the representation without the standard reification as the control.

Hypergraph neural networks. Our encoder lies in the design space of HGNN, UniGNN, AllSet and multi-relational hypergraph message passing (Feng et al., 2019; Huang and Yang, 2021; Chien et al., 2022; Yadati, 2020); see Kim et al. (2024) for a taxonomy. We do not propose a new architecture; the comparison only requires that both arms use the same one.

3 Representations and encoder

3.1 Three views of the same fact

A fact f has a relation r_f and members $\{(v_1, \rho_1), \ldots, (v_k, \rho_k)\}$, where ρ_i is the role of entity v_i (subject, object, or a qualifier property). Figure 1 shows the two views compared throughout.

- **preserve** (hyperedge). f is one hyperedge of type r_f whose incidences carry the roles ρ_i. All members meet in one message-passing hop.
- **reify** (standard reification). f becomes a statement node b_f, and each member becomes a binary edge (b_f, v_i) whose *type* encodes ρ_i; the incidences of these edges carry only the generic roles subject and object. The statement node carries r_f, as the type triple of classic reification does. No information is lost, since f can be read back from b_f and its edges, but members are now two hops apart.
- **prune** (Study 3 only). f is a hyperedge over its subject and object; qualifiers are absent from the encoder graph. In Study 3 the decoder still receives the qualifier tokens, so this arm isolates what qualifier *structure in the encoder* adds.

3.2 Role-aware hypergraph encoder

The encoder of Studies 1 and 3 alternates node-to-edge and edge-to-node updates. With E_{role} a role embedding and $\phi_{1..4}$ linear maps, each layer computes

$$m_e = \text{mean}_{v \in e}\ \sigma(\phi_1(h_v + E_{\text{role}}[\rho_{v,e}]))\,, \qquad h_e \leftarrow \text{LN}(h_e + \phi_2([h_e; m_e]))\,, \tag{1}$$

$$m_v = \text{mean}_{e \ni v}\ \sigma(\phi_3(h_e + E_{\text{role}}[\rho_{v,e}]))\,, \qquad h_v \leftarrow \text{LN}(h_v + \phi_4([h_v; m_v]))\,, \tag{2}$$

with GELU σ and layer normalisation LN; the edge-to-node step uses the updated h_e, and two layers are stacked. Edges start from a relation embedding, $h_e^0 = E_{\text{rel}}[r_e]$, and node states pass through an input layer normalisation. The same module consumes either view; only the incidence list differs. In the reified view the statement node is an ordinary node and its binary edges are ordinary two-member hyperedges.

In Study 1, nodes start from a type embedding plus a projected text embedding, and completion scores candidate nodes against a query built from the fact's hidden state and the masked role. The fact embedding is aligned with sentence embeddings of the fact by CLIP-style contrastive learning (Radford et al., 2021), which defines the retrieval task. preserve reads the fact through the edge head $W_e h_e$. In the baseline and in conditions A and C the reified arm reads h_{b_f} through the node head; condition B, and with it D, the combined condition and the depth condition, applies the same edge head to h_{b_f}. The default text tower is a hashed n-gram encoder; condition C replaces it with the frozen sentence encoder paraphrase-multilingual-MiniLM-L12-v2 (Reimers and Gurevych, 2019). A structure-free reference line (flat_text) encodes only the fact sentence through an MLP of matching size; it is reported but never judged.

In Study 3 the encoder replaces StarE's convolution: nodes start from StarE's learned entity table, a statement node starts from its relation's edge embedding plus one shared learned vector, StarE's dropout is applied after each layer, and StarE's Transformer decoder reads the output node table unchanged.

3.3 Fairness conditions

In Study 1 both arms share the module, the vocabulary, the parameter count (351,553), the per-seed initial weights, the budget and the text encoder; automated tests pin these conditions. In Study 3 the three arms share every module and differ in parameter count only by the 200-dimensional shared vector of reify (5,318,216 vs. 5,318,416). In Study 2 the native StarE encoder has 1.6% more parameters than the reified arm, because the reified graph needs no qualifier aggregation weights (5,068,016 vs. 4,989,016).

4 Experimental design

4.1 Data

The synthetic world has 78 entities and 390 n-ary facts with template sentences (272 training, 59 validation and 59 test facts), which define both a retrieval task (find a fact from a sentence) and a completion task. WD50K(100) is the subset of the StarE release in which every statement has at least one qualifier (Galkin et al., 2020): 31,314 statements (22,738/3,279/5,297), 18,791 entities across all positions, of which 7,917 appear only as qualifier values, 204 predicates and 134 qualifier properties. Study 1's loader keeps the first value of a repeated qualifier property

Table 1: Design of the three studies. All judged comparisons use seeds 1–5.

	Study 1	Study 2	Study 3
Encoder	role-aware hypergraph encoder ($d = 64$, 2 layers)	StarE, unmodified files ($d = 200$)	role-aware hypergraph encoder ($d = 200$, 2 layers)
Decoder	completion query · node embeddings	StarE Transformer	StarE Transformer
Arms	preserve vs. reify	native graph vs. reify	preserve vs. reify; preserve vs. prune
Data	synthetic world; WD50K(100)	WD50K(100)	WD50K(100)
Protocol	own (any role masked, all entities, filtered; WD50K: 2,000 sampled test facts)	StarE standard (subject/object, filtered, full test set)	StarE standard
Budget	1,500 steps, batch 32	401 epochs	401 epochs
Judged metrics	recall@5 and acc@1 (synthetic); MRR and Hits@10 (WD50K)	test MRR and Hits@10 at epoch 401	test MRR and Hits@10 at epoch 401

within a statement, giving 18,025 entities and mean arity 3.30; Studies 2 and 3 use StarE's own loader, which keeps repeats and caps a statement at six qualifier pairs.

4.2 Three studies

Table 1 summarises the design. Studies 2 and 3 follow StarE's standard protocol exactly: the subject or object of each test statement is masked (both directions are evaluated), candidates are all entities except those that appear only as qualifier values, ranking is filtered (Bordes et al., 2013), the graph contains train and validation statements, and training runs StarE's default budget (401 epochs, batch 128, Adam with learning rate 10^{-4}, label smoothing 0.1). Study 1 uses its own protocol, described in Section 5.4.

4.3 Positioning run: HAHE on the same pipeline

The published WD50K(100) values of the strongest models exist only as a figure, and a gap of a few thousandths of MRR could come from protocol differences alone. We therefore ran HAHE (Luo et al., 2023), which combines hypergraph attention over a global graph of training facts (by its default, the first fifth of them) with a 12-layer Transformer over each fact, on the pipeline of Studies 2 and 3. Its five source files are imported unmodified at a pinned commit (`e35e0a7b`) and fed the rows produced by StarE's loader, so the training graph, the 10,594 test queries, the 10,874 candidate entities and the filter are those of Studies 2 and 3; each StarE entity maps to one HAHE token, and automated tests check the mapping. HAHE scores its whole vocabulary; we take the entity scores and rank them with StarE's rule, which our tests check against StarE's formula on random scores.

Training follows HAHE's published WD50K recipe without tuning (hidden size 256, AdamW with a one-cycle schedule, batch 512, 300 epochs; no WD50K(100) recipe exists). HAHE's objective masks every position of a fact—subject, relation, object, qualifier properties and values—which gives 298 optimiser steps per epoch, and the model has 26,584,063 parameters, five times the 5,318,216 of our encoder. We compare HAHE at epoch 300 with preserve of Study 3 at epoch 401. Epochs are not matched, because the two models train on different queries and an epoch is not the same amount of training; we report steps, compute and the learning curve instead. As a check on the port, we also score HAHE with its own ranking rule (all 18,791 entities as

candidates, ties counted against the target) and compare it with the GLoRE authors' re-run (Ji et al., 2026); agreement within ±0.03 was fixed in advance as the criterion, reported but not judged.

4.4 Judging rule and pre-registration

A seed is won by the arm that is strictly better on it; ties count for nobody. An arm wins a comparison only if it wins *both* judged metrics on at least 4 of 5 seeds; otherwise the result is "no consistent order". In Study 1 the rule was one-sided (does preserve beat reify?). In Studies 2 and 3 and in the HAHE comparison it was two-sided; our predictions, recorded before the runs, favoured reify in Study 2 (55%), preserve in Study 3 (60%) and HAHE over our encoder (50%, against 30% for no consistent order). Arms, metrics, budgets and seeds were never changed after results were seen.

The plans were written in versioned documents ("charters"). How far an outsider can verify their timing differs by study, and none of them sits on a third-party registry. Study 1's baseline plan was committed together with its result, and the v2 charter covering conditions A–D and the combined condition was committed before those conditions were implemented (commit `c054b59d5`, 20 August 2026; results 22 August), after the baseline's direction was known. The depth condition's charter is dated before its run but was first committed together with its results. For Study 2, the judged-run rule entered the repository's event log before the first judged run finished and was committed with the results; for Study 3 and for the HAHE comparison of Section 5.4, the charter and event-log entries precede the runs and are committed with the results. The statistical tests of Section 5.1 were planned after the Study 1 verdicts but committed before any test statistic was computed (commit `24b2e1f71`).

Beyond the rule we report seed-paired differences with 95% confidence intervals and two-sided t-tests (4 degrees of freedom), and an exact sign-flip test. Table 3 reports preserve − control, so positive values favour preserve; Table 4 reports control − preserve, so negative values favour preserve. With five seeds the smallest attainable sign-flip p is $2/32 = 0.0625$, so that test confirms only that all seeds agree in direction. Reporting seed variation follows Bouthillier et al. (2021).

4.5 Implementation

All runs used one 8 GB RTX 3060 Ti. Study 1 ran in PyTorch's deterministic mode. Studies 2 and 3 use StarE at commit `b294b9e2` with PyTorch 2.10 and PyTorch Geometric 2.6.1; the three `torch_scatter` functions StarE uses are replaced by native PyTorch equivalents, checked against explicit loops. To avoid hardware resets under sustained load, the runs idle the GPU for 150 ms after each optimiser step and checkpoint every five epochs; the idle time does not change the computation. One Study 3 run (reify, seed 1) was resumed from its epoch-30 checkpoint after a machine reset. The HAHE runs use the same schedule and also checkpoint the optimiser, learning-rate schedule, random-number and data-shuffling states; two of them were resumed, one after an application restart and one after a machine reset, and in both the training loss of the first resumed epoch matched the value recorded before the interruption.

Table 2: Study 1. Means over five seeds; the control is reify in every row (its four-layer variant in the last row). In every condition preserve wins both metrics on 5/5 seeds.

Condition	What is controlled	Metrics	preserve	control
Baseline	—	recall@5 / acc@1	**0.863 / 0.502**	0.670 / 0.424
A: inductive	evaluated facts' structure removed from training	recall@5 / acc@1	**0.870 / 0.310**	0.723 / 0.212
B: head parity	control reads through the same output head	recall@5 / acc@1	**0.863 / 0.502**	0.699 / 0.403
C: frozen text	frozen multilingual sentence encoder	recall@5 / acc@1	**0.846 / 0.405**	0.585 / 0.251
D: real data	WD50K(100), inductive, matched head	MRR / Hits@10	**0.147 / 0.289**	0.118 / 0.215
A+B+C (reported)	three synthetic controls together; pre-specified as report-only	recall@5 / acc@1	**0.815 / 0.234**	0.605 / 0.178
Depth + parity	control has 4 layers, +14% parameters, matched head	recall@5 / acc@1	**0.863 / 0.502**	0.672 / 0.400

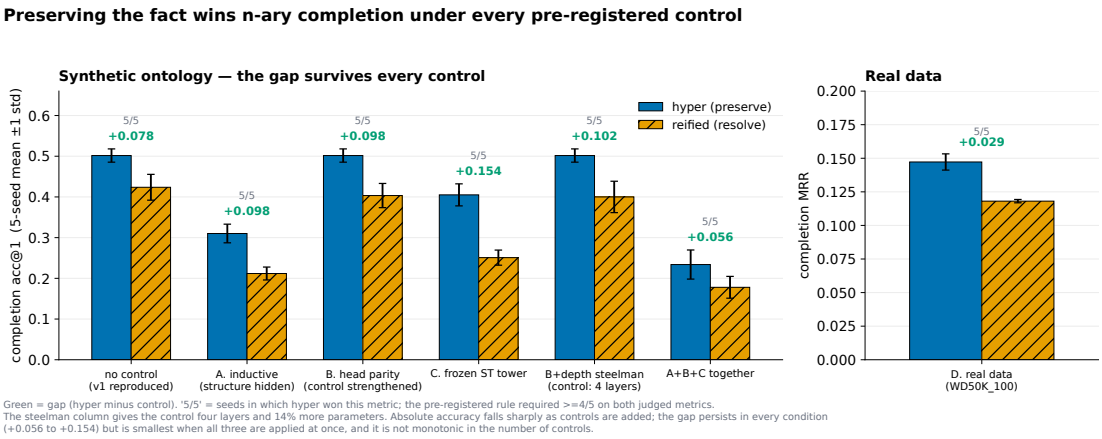


Figure 2: Study 1: completion per condition (mean over five seeds, ±1 s.d.). Absolute accuracy falls as controls are added, but a gap remains in every condition (+0.056 to +0.154). The right panel is WD50K(100) with MRR, on its own axis.

5 Results

5.1 Study 1: one encoder, seven conditions

Table 2 lists the seven conditions. A, B and C each change one thing relative to the baseline. D uses real data under the inductive protocol with the matched head. The depth condition changes only depth relative to B, and the report-only combined condition applies A, B and C together. preserve won both metrics on 5 of 5 seeds in every condition (Figure 2).

Three readings follow. First, the controls make the task harder: completion accuracy falls from 0.502 to 0.310 under the inductive protocol and to 0.234 with three controls. Second, the gap does not shrink with it; under the inductive protocol it grows from +0.078 to +0.098. Third, two alternative explanations fail in this synthetic setting. Matching the control's output head raised its retrieval slightly and lowered its completion, leaving the gap in place. Giving the control twice the layers and 14% more parameters, together with the matched head, *widened* the completion gap from +0.098 to +0.102; the charter had fixed, before the run, that a reduction below 25% would mean depth does not explain the gap. Depth alone (without head parity) narrowed it by only 9% (+0.078 to +0.071).

Table 3: Study 1: seed-paired tests (difference = preserve − control; two-sided t, 4 d.f.; Holm correction across the 14 pairs).

Condition	Metric	Mean difference [95% CI]	$t(4)$	p (Holm)	d_z
Baseline	recall@5	+0.193 [+0.121, +0.265]	7.43	0.016	3.32
Baseline	acc@1	+0.078 [+0.046, +0.110]	6.78	0.020	3.03
A: inductive	recall@5	+0.147 [+0.100, +0.194]	8.67	0.010	3.88
A: inductive	acc@1	+0.098 [+0.058, +0.139]	6.74	0.020	3.02
B: head parity	recall@5	+0.164 [+0.095, +0.233]	6.57	0.020	2.94
B: head parity	acc@1	+0.098 [+0.079, +0.117]	14.28	0.002	6.39
C: frozen text	recall@5	+0.262 [+0.064, +0.459]	3.67	0.043	1.64
C: frozen text	acc@1	+0.154 [+0.127, +0.181]	15.72	0.001	7.03
D: real data	MRR	+0.029 [+0.022, +0.036]	11.97	0.003	5.35
D: real data	Hits@10	+0.074 [+0.065, +0.083]	22.26	<0.001	9.96
A+B+C	recall@5	+0.210 [+0.089, +0.332]	4.80	0.033	2.15
A+B+C	acc@1	+0.056 [+0.003, +0.109]	2.93	0.043	1.31
Depth + parity	recall@5	+0.191 [+0.101, +0.282]	5.89	0.021	2.63
Depth + parity	acc@1	+0.102 [+0.044, +0.160]	4.86	0.033	2.17

Table 3 gives the seed-paired tests. All 14 pairs remain significant after Holm correction (Holm, 1979), and all have sign-flip $p = 0.0625$. Two are borderline: completion under A+B+C (confidence interval lower bound +0.003) and retrieval under C (Holm $p = 0.043$, wide interval). In the hardest condition the direction is consistent but the size cannot be distinguished from values near zero, and we rest no claim on it.

5.2 Study 2: an external encoder with unmodified code

Table 4 (upper block) gives the result. StarE's native graph beat the reified graph on both metrics on 5 of 5 seeds: test MRR 0.6553 vs. 0.6485, paired difference −0.0068 (95% CI −0.0104 to −0.0031, $p = 0.0067$), and Hits@10 0.7766 vs. 0.7708 (−0.0058, CI −0.0073 to −0.0043). The native arm reproduces the published 0.654 (Galkin et al., 2020) to within 0.002, so the port is faithful.

The learning curves (Figure 3a) show why the budget matters. Averaged over seeds, the reified graph leads at epochs 10–20 and again from epoch 60 to epoch 110, by about 0.09 MRR at 70–80 epochs; the native graph leads briefly at 30–50 epochs and then from epoch 120 to the end. A pilot planned before the judged runs (one seed, 76 epochs, training graph only, validation set) had recorded the reified graph ahead at 0.434 vs. 0.336—the opposite of the converged verdict. We report both because both are true at their budgets.

5.3 Study 3: our encoder under the standard protocol

Both comparisons favoured preserve on all five seeds and on every reported metric (Table 4, lower block). Against reify the MRR gap is −0.0087 ($p = 0.0004$); against prune it is −0.0129 ($p = 0.0010$). Hits@1, Hits@3 and mean rank point the same way on 5/5 seeds for both controls. The reified arm keeps pace with preserve for the first 70 epochs, leading only at epochs 30–40 on the seed mean, and falls behind after about 100 (Figure 3b); at the final epoch preserve is the best of the three arms on every seed. Removing qualifiers from the encoder slows learning markedly (mean MRR 0.469 vs. 0.587 at epoch 50) and the gap persists after convergence, even though the decoder of all three arms receives the qualifier tokens.

Table 4: Studies 2 and 3: WD50K(100), StarE standard protocol, test set, epoch 401, mean ± s.d. over seeds 1–5. Paired difference = control − preserve (or native StarE graph); negative values favour preserve. Per-seed values are in Appendix A.

	Arm	MRR	Hits@1	Hits@10	Seeds won
Study 2 (StarE)	native graph	**0.6553 ± 0.0013**	**0.5891**	**0.7766**	5/5, 5/5
	reify	0.6485 ± 0.0021	0.5809	0.7708	—
Study 3 (ours)	preserve	**0.6648 ± 0.0026**	**0.6025**	**0.7777**	5/5, 5/5 (both)
	reify	0.6561 ± 0.0024	0.5942	0.7686	—
	prune	0.6519 ± 0.0026	0.5871	0.7709	—
Paired differences (MRR; Hits@10), 95% CI					
Study 2: reify − native		−0.0068 [−0.0104, −0.0031];		−0.0058 [−0.0073, −0.0043]	
Study 3: reify − preserve		−0.0087 [−0.0110, −0.0065];		−0.0092 [−0.0106, −0.0077]	
Study 3: prune − preserve		−0.0129 [−0.0171, −0.0087];		−0.0069 [−0.0107, −0.0031]	

5.4 Position among published models

Table 5 places Studies 2 and 3 among published results on WD50K(100) under the same protocol. Our encoder sits at the level of StarE; its 0.0095 margin over StarE is not a judged comparison and comes with 4.9% more parameters, so we do not claim it. HAHE and MAYPL did not report WD50K(100); the GLoRE authors re-ran both, and their Figure 5 places HAHE at about 0.670 and MAYPL at about 0.658—on either side of our encoder—and GLoRE at about 0.824, far above all of them (Ji et al., 2026). A gap of about 0.005 is too small to order models whose protocols may differ, which is why we ran HAHE on our pipeline. Study 1's WD50K number (0.147) does not belong in this table: it masks any role including qualifier values, ranks all 18,025 entities of its loader, trains for 1,500 steps of 32 facts at $d = 64$ rather than about 17.5 million row updates at $d = 200$, and scores a 2,000-fact sample.

HAHE on the same pipeline. HAHE beat our encoder on both judged metrics on all five seeds (Table 6): MRR +0.0135 (95% CI +0.0103 to +0.0168) and Hits@10 +0.0170 (+0.0130 to +0.0210). The port reproduces the literature: under HAHE's own ranking rule its mean MRR is 0.6778, +0.008 from the GLoRE re-run. Two details qualify the lead. It comes from subject prediction (+0.0305 MRR, 5/5 seeds); on object prediction our encoder is ahead on four of five seeds (−0.0034). And HAHE's mean rank is worse on every seed (411.9 vs. 357.3): it ranks more answers near the top but pushes the answers it misses further down. The unmatched budgets do not explain the result, since HAHE's test MRR passes our encoder's final value by epoch 20 and changes little after epoch 50. HAHE differs from our encoder in size, in its fact-level Transformer and in its training objective, and we did not separate these. HAHE also keeps each fact whole, so its lead bears on the encoder, not on preservation versus reification; whether HAHE loses accuracy on the reified graph is untested. Under an identical pipeline our encoder sits below HAHE. It is a vehicle for a controlled comparison, not a competitor.

5.5 Cost

Reification enlarges the graph. On WD50K(100) the reified graph used 1.7× the GPU memory in Study 1 (970 vs. 569 MB), 1.5× in Study 2 (1,537 vs. 1,022 MB) and 2.2× in Study 3 (2,850 vs. 1,309 MB), and in Study 3 it took 30% longer per epoch (42.8 vs. 33.0 s). In Study 2 the reified arm was faster (36.1 vs. 47.2 s per epoch) because StarE's native encoder spends time

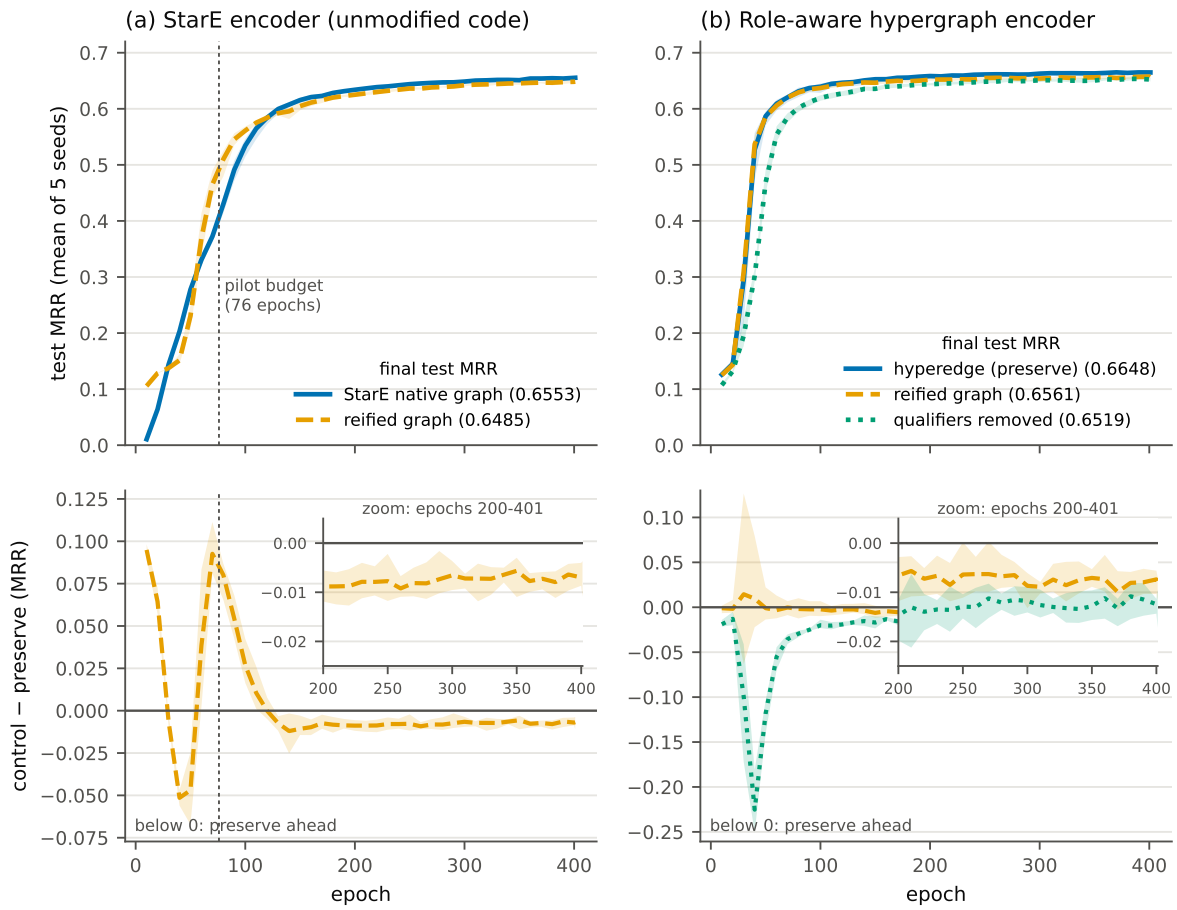


Figure 3: Test MRR during training under the standard protocol (mean of five seeds; bands span the seeds). Top: MRR. Bottom: paired difference from the preserved or native graph; the insets zoom into epochs 200–401. (a) Study 2: the reified graph leads at most evaluation points before epoch 120, apart from a brief native lead at 30–50 epochs; the dotted line marks the budget of the pilot, which used the training graph and the validation set and recorded the opposite of the converged order. (b) Study 3: the reified arm tracks preserve closely early and falls behind after about 100 epochs; removing qualifiers from the encoder slows learning sharply and leaves a lasting gap.

on qualifier aggregation that the reified arm does not need. HAHE is heavier than our encoder: five times the parameters, 4,023 vs. 1,309 MB of GPU memory, and 114.3 vs. 33.0 s of training compute per epoch, about 2.6 times the compute per run despite its shorter budget.

Table 5: WD50K(100), subject/object prediction, filtered, test set.

Model	MRR	Hits@10	Source
Transformer (T), no qualifiers	0.351	0.530	Galkin et al. (2020)
StarE (T), no qualifiers	0.380	0.584	Galkin et al. (2020)
NaLP-Fix	0.458	0.563	Galkin et al. (2020)
HINGE	0.492	0.636	Galkin et al. (2020)
Transformer (H)	0.562	0.677	Galkin et al. (2020)
StarE (H)	0.654	0.777	Galkin et al. (2020)
StarE native graph, our run (5 seeds)	0.6553	0.7766	Study 2
StarE reified graph (5 seeds)	0.6485	0.7708	Study 2
Our encoder, prune (5 seeds)	0.6519	0.7709	Study 3
Our encoder, reify (5 seeds)	0.6561	0.7686	Study 3
Our encoder, preserve (5 seeds)	0.6648	0.7777	Study 3
HAHE, our run on the same pipeline (5 seeds)[b]	0.6784	0.7947	This paper
MAYPL, re-run by Ji et al. (2026)[a]	≈0.658	—	Ji et al. (2026), Fig. 5(a)
HAHE, re-run by Ji et al. (2026)[a]	≈0.670	—	Ji et al. (2026), Fig. 5(a)
GLoRE[a]	≈0.824	—	Ji et al. (2026), Fig. 5(a)

[a] The GLoRE paper reports WD50K(100) only as a figure. We read the marker positions of Figure 5(a) (main entities) from the page rendered at 400 dpi against its grid lines; the read values reproduce the figure's printed margin of GLoRE over HAHE (+0.153, +23%) to within 0.001, and we estimate the reading error at about ±0.003.

[b] HAHE's public code at a pinned commit, unmodified, with its published WD50K hyperparameters (not tuned) for 300 epochs, on the same rows, queries, candidates and scoring as Studies 2 and 3 (Section 4.3); judged against preserve in Table 6. Under HAHE's own ranking (all 18,791 entities as candidates, ties counted against the target) the mean MRR is 0.6778.

Table 6: Our encoder (preserve, epoch 401) and HAHE (epoch 300) on the identical pipeline: WD50K(100), StarE's ranking rule, test set, mean over seeds 1–5. Difference = HAHE − preserve. MRR and Hits@10 are judged; the other rows are reported. Object and subject MRR split the test queries by which position is masked. Per-seed values are in Appendix A.

	preserve	HAHE	Difference [95% CI]	HAHE ahead
MRR	0.6648 ± 0.0026	**0.6784 ± 0.0017**	+0.0135 [+0.0103, +0.0168]	5/5
Hits@10	0.7777 ± 0.0017	**0.7947 ± 0.0028**	+0.0170 [+0.0130, +0.0210]	5/5
Hits@1	0.6025	**0.6152**	+0.0127	5/5
Mean rank[a]	**357.3**	411.9	+54.6	0/5
Object MRR	**0.6757**	0.6723	−0.0034	1/5
Subject MRR	0.6539	**0.6844**	+0.0305	5/5
Parameters	5,318,216	26,584,063		

[a] Lower is better; HAHE's mean rank is worse on every seed.

6 Discussion

What the evidence supports. The same direction appeared in all nine judged comparisons and in the report-only condition, with two encoders, two protocols and two budgets, on 5/5 seeds each time. Under the standard protocol and budget the gaps are small (0.007–0.013 MRR) and their size is not the evidence; their consistency is. We read the result as: *for the two message-passing encoders we tested, the standard reification of an n-ary fact is a slightly worse input than the fact itself; in Study 1's synthetic setting the difference did not come from the output head or from effective depth, and across studies it did not come from the text encoder or from a short budget.*

Why preservation helps. In the reified graph, members of a fact meet only through the statement node, two hops apart, and the fact's state is an ordinary node that is updated only after a further layer. The depth condition shows that the number of hops alone is not the reason: four layers did not close the gap in Study 1. In Study 3 both preserve and reify let qualifier values receive messages, so "qualifier values are updated" is not the reason either. The difference that remains is where the fact's state lives: as an edge state updated jointly with its members (preserve), or as an ordinary node competing with entity nodes for the same message-passing capacity (reify). We offer this as a hypothesis; none of our studies isolates it.

Budget and the direction of comparisons. With StarE's encoder, short budgets favoured the reified graph: it led at most evaluation points before epoch 120, by about 0.09 MRR at 70–80 epochs. With our encoder the reified arm led only at epochs 30–40, and in Study 1 preserve led by a wide margin at a short budget. The early advantage is therefore a property of the StarE pairing, not of reification in general. It still matters for anyone comparing representations: a comparison stopped early can report the opposite of the converged order.

Does qualifier structure matter at all? Wang et al. (2025) report that a qualifier-free NBFNet matches or exceeds hyper-relational models on WD50K. In Study 3 removing qualifiers from the encoder lost on all five seeds even though the decoder still saw them. The two results are not in conflict: WD50K(100) has qualifiers on every statement while WD50K has them on 13.6%, and the models differ. We conclude only that, on a dataset where qualifiers are dense, encoder-side qualifier structure is not replaced by decoder-side qualifier tokens.

What we do not claim. *Retrieval.* In Study 1, the structure-free flat_text line beat preserve at retrieval under the frozen sentence encoder (recall@5 0.897 vs. 0.846), as the charter had predicted, while losing badly at completion (0.236 vs. 0.405). Finding a fact and binding its participants are different abilities (Figure 4); recall@5 served only to compare the two structured representations. *Accuracy.* The encoder is not a state-of-the-art model: on the identical pipeline it is below HAHE on all five seeds, and it is far below GLoRE (Table 5); only the HAHE comparison is judged, and the rest of the position is descriptive. *Generality.* Two encoders and two datasets (one synthetic, one Wikidata) do not support "on any backbone" or "on any knowledge graph". *Language models.* We also pre-registered an attempt to inject the encoder's states into a frozen decoder-only language model (Qwen Team, 2024) as an alternative to retrieval-augmented generation (Lewis et al., 2020). All six cycles failed: the best arm reached

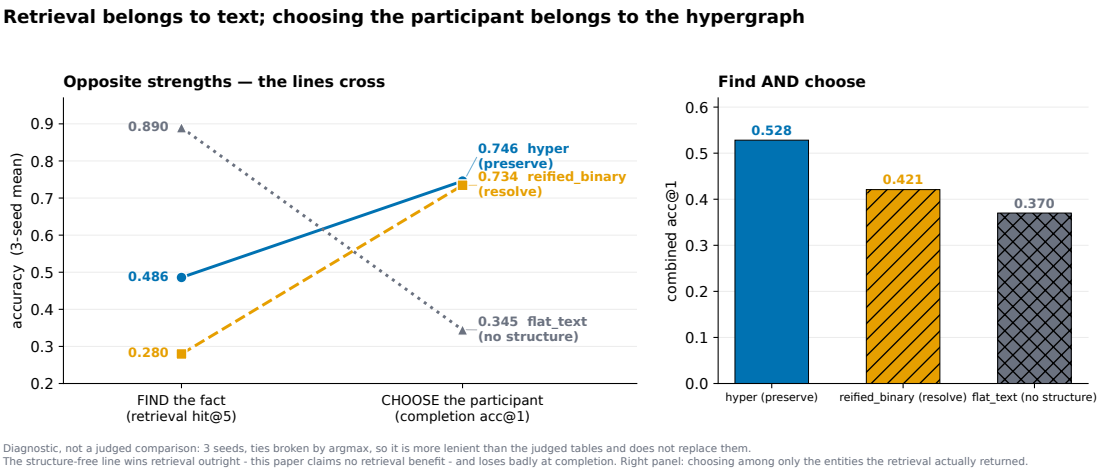


Figure 4: Finding a fact and choosing its participants are different abilities (Study 1 diagnostic, three seeds, ties broken by argmax; not a judged result). The structure-free text encoder ranks first at retrieval and last at completion; preserve ranks first at completion and second at retrieval.

0.244 on multi-constraint questions, below a plain list of the retrieved entity names (0.400). The advantage measured here is an encoder-level property and stops there.

Limitations. (i) Real data is a single dataset, WD50K(100), where every statement has qualifiers—the setting most favourable to qualifier structure. (ii) Studies 2 and 3 hold the budget and the hyperparameters at StarE's defaults, which StarE's authors tuned for its native graph; no arm was tuned, and how the converged gap scales with model size is untested. (iii) Study 2's gap (0.007 MRR) could in part reflect the native arm's 80,000 extra qualifier-aggregation weights (1.6% of its parameters), which we did not control; Study 3's arms have 4.9% more parameters than StarE, so the cross-study position in Table 5 is descriptive. (iv) Our encoder keeps one representation per entity, the node-level design that GLoRE's authors report confuses roles; a pair-level encoder might show a different gap. (v) Tests are at the seed level with $n = 5$; per-query predictions were not stored, so query-level tests were not run. (vi) Plans are documented in the repository rather than on a third-party registry, and for several of them the evidence that they preceded the results is the repository's own event log (Section 4.4). (vii) We ran HAHE only with its published WD50K hyperparameters, set for the full WD50K rather than WD50K(100), and for 300 epochs against our encoder's 401 (its test MRR passes our encoder's final value by epoch 20 and is flat after epoch 50). We did not run MAYPL, KREPE or GLoRE; their WD50K(100) results exist only as a figure in the GLoRE paper, from which we read them, and the MAYPL value is that paper's re-run.

7 Conclusion

Holding the encoder, its budget and its information fixed, and its parameters fixed or within 1.6% (except the deliberately larger control of the depth condition), keeping an n-ary fact whole—as a hyperedge, or as StarE's qualifier-carrying edge—was a better input than its standard reification in all nine judged comparisons, on two encoders and under two protocols. Under the standard protocol the advantage is about one point of MRR and is in place by epoch 120 of 401 in both studies; with StarE's encoder, reification learns faster early and can look better at short budgets. The practical advice is modest: if a model can take n-ary facts directly, giving it the reified form

costs a little accuracy and more memory, and representation comparisons should be judged at convergence. The evidence concerns the input representation, not our encoder as a model: on an identical pipeline HAHE beats it on all five seeds.

Data and code availability

Code (including the bridge that runs HAHE's pinned public code on the StarE pipeline), run configurations, per-epoch metrics, judgment files and the plan of each study (in Korean) are available at `https://github.com/booobing/preserve-do-not-reify`. WD50K(100) is part of the StarE release (Galkin et al., 2020). Every number in this manuscript is checked against the run artifacts by a script in the repository.

Declaration of generative AI use

[The authors must complete this statement truthfully.] During the preparation of this work the author(s) used Claude (Anthropic) to write and debug experiment code, run analyses, and draft and edit the text. After using this tool, the author(s) reviewed and edited the content and take full responsibility for the publication.

References

Antoine Bordes, Nicolas Usunier, Alberto Garcia-Durán, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems 26 (NIPS 2013)*, 2013.

Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Nazanin Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems (MLSys 2021)*, 2021.

Eli Chien, Chao Pan, Jianhao Peng, and Olgica Milenkovic. You are AllSet: A multiset function framework for hypergraph neural networks. In *International Conference on Learning Representations (ICLR 2022)*, 2022.

Richard Cyganiak, David Wood, and Markus Lanthaler. RDF 1.1 concepts and abstract syntax. W3C Recommendation, 25 February 2014. `https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/`, 2014.

Bahare Fatemi, Perouz Taslakian, David Vazquez, and David Poole. Knowledge hypergraphs: Prediction beyond binary relations. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI 2020)*, 2020. arXiv:1906.00137.

Yifan Feng, Haoxuan You, Zizhao Zhang, Rongrong Ji, and Yue Gao. Hypergraph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI 2019)*, 2019.

Mikhail Galkin, Priyansh Trivedi, Gaurav Maheshwari, Ricardo Usbeck, and Jens Lehmann. Message passing for hyper-relational knowledge graphs. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*, pages 7346–7359, 2020. doi: 10.18653/v1/2020.emnlp-main.596.

Saiping Guan, Xiaolong Jin, Yuanzhuo Wang, and Xueqi Cheng. Link prediction on n-ary relational data. In *Proceedings of The World Wide Web Conference (WWW 2019)*, pages 583–593, 2019. doi: 10.1145/3308558.3313414.

Saiping Guan, Xiaolong Jin, Jiafeng Guo, Yuanzhuo Wang, and Xueqi Cheng. Link prediction on n-ary relational data based on relatedness evaluation. arXiv:2104.10424, 2021.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Zhiwei Hu, Víctor Gutiérrez-Basulto, Zhiliang Xiang, Ru Li, and Jeff Z. Pan. HyperFormer: Enhancing entity and relation interaction for hyper-relational knowledge graph completion. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM 2023)*, pages 803–812, 2023. doi: 10.1145/3583780.3614922.

Jing Huang and Jie Yang. UniGNN: a unified framework for graph and hypergraph neural networks. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI 2021)*, 2021.

Sangjun Ji, Sangjune Kim, Youngho Lee, Bonyou Koo, Xiongnan Jin, and Byungkook Oh. Joint global-local representations via relation-entity pair encoding for hyper-relational knowledge graphs. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026)*, pages 2050–2061. ACM, 2026. doi: 10.1145/3770855.3818000.

Sunwoo Kim, Soo Yong Lee, Yue Gao, Alessia Antelmi, Mirko Polato, and Kijung Shin. A survey on hypergraph neural networks: An in-depth and step-by-step guide. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2024)*, 2024. doi: 10.1145/3637528.3671457.

Jaejun Lee and Joyce Jiyoung Whang. Structure is all you need: Structural representation learning on hyper-relational knowledge graphs. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, volume 267 of *PMLR*, pages 33765–33783, 2025.

Jaejun Lee, Seheon Kim, and Joyce Jiyoung Whang. Generative representation learning on hyper-relational knowledge graphs via masked discrete diffusion. In *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, volume 306 of *PMLR*, 2026. arXiv:2605.24064.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

Haoran Luo, Haihong E, Yuhao Yang, Yikai Guo, Mingzhi Sun, Tianyu Yao, Zichen Tang, Kaiyang Wan, Meina Song, and Wei Lin. HAHE: Hierarchical attention for hyper-relational knowledge graphs in global and local level. In *Proceedings of the 61st Annual Meeting of*

the Association for Computational Linguistics (Volume 1: Long Papers) (ACL 2023), pages 8095–8107, 2023. doi: 10.18653/v1/2023.acl-long.450.

Natasha Noy and Alan Rector. Defining n-ary relations on the semantic web. W3C Working Group Note, 12 April 2006. https://www.w3.org/TR/swbp-n-aryRelations/, 2006.

Qwen Team. Qwen2.5 technical report. arXiv:2412.15115, 2024.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)*, volume 139 of *PMLR*, pages 8748–8763, 2021.

Rajesh Rajagopalamenon and Unnikrishnan Cheramangalath. HEHRGNN: A unified embedding model for knowledge graphs with hyperedges and hyper-relational edges. arXiv:2602.18897, 2026.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP 2019*, pages 3982–3992, 2019.

Paolo Rosso, Dingqi Yang, and Philippe Cudré-Mauroux. Beyond triplets: Hyper-relational knowledge graph embedding for link prediction. In *Proceedings of The Web Conference 2020 (WWW 2020)*, 2020. doi: 10.1145/3366423.3380257.

Shikhar Vashishth, Soumya Sanyal, Vikram Nitin, and Partha Talukdar. Composition-based multi-relational graph convolutional networks. In *International Conference on Learning Representations (ICLR 2020)*, 2020.

Denny Vrandečić and Markus Krötzsch. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014. doi: 10.1145/2629489.

Quan Wang, Haifeng Wang, Yajuan Lyu, and Yong Zhu. Link prediction on n-ary relational facts: A graph-based approach. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 396–407, 2021. doi: 10.18653/v1/2021.findings-acl.35.

Yubo Wang, Shimin Di, Zhili Wang, Haoyang Li, Fei Teng, Hao Xin, and Lei Chen. Understanding the embedding models on hyper-relational knowledge graph. arXiv:2508.03280, 2025.

Jianfeng Wen, Jianxin Li, Yongyi Mao, Shini Chen, and Richong Zhang. On the representation and embedding of knowledge bases beyond binary relations. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI 2016)*, pages 1300–1307, 2016.

Naganand Yadati. Neural message passing for multi-relational ordered and recursive hypergraphs. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

Jixiao Zhang, Yongkang Li, Ruotong Zou, Jingyuan Zhang, Zipei Fan, and Xuan Song. Hyper-relational knowledge graph neural network for next POI recommendation. arXiv:2311.16683, 2023.

Zhaocheng Zhu, Zuobai Zhang, Louis-Pascal Xhonneux, and Jian Tang. Neural Bellman-Ford networks: A general graph neural network framework for link prediction. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021.

A Per-seed results of Studies 2 and 3 and of the HAHE comparison

Table 7: Test MRR / Hits@10 at epoch 401 for each seed.

	Study 2 (StarE)		Study 3 (our encoder)		
Seed	native	reify	preserve	reify	prune
1	0.6551 / 0.7751	0.6468 / 0.7702	0.6657 / 0.7803	0.6578 / 0.7725	0.6519 / 0.7698
2	0.6566 / 0.7753	0.6501 / 0.7705	0.6651 / 0.7768	0.6559 / 0.7664	0.6556 / 0.7731
3	0.6541 / 0.7786	0.6504 / 0.7714	0.6657 / 0.7785	0.6588 / 0.7699	0.6485 / 0.7687
4	0.6568 / 0.7760	0.6458 / 0.7691	0.6604 / 0.7767	0.6525 / 0.7662	0.6510 / 0.7712
5	0.6541 / 0.7777	0.6496 / 0.7727	0.6672 / 0.7764	0.6556 / 0.7679	0.6527 / 0.7715

Table 8: HAHE comparison per seed: preserve at epoch 401 and HAHE at epoch 300. Columns 2–3 give MRR / Hits@10 under StarE's rule; column 4 is HAHE's MRR under its own rule; columns 5–6 give MRR by prediction direction (preserve / HAHE).

Seed	preserve	HAHE	HAHE, own rule	Object	Subject
1	0.6657 / 0.7803	0.6775 / 0.7974	0.6769	0.6726 / 0.6706	0.6589 / 0.6844
2	0.6651 / 0.7768	0.6770 / 0.7945	0.6765	0.6760 / 0.6719	0.6542 / 0.6822
3	0.6657 / 0.7785	0.6777 / 0.7902	0.6771	0.6789 / 0.6708	0.6525 / 0.6845
4	0.6604 / 0.7767	0.6783 / 0.7948	0.6776	0.6721 / 0.6728	0.6487 / 0.6837
5	0.6672 / 0.7764	0.6813 / 0.7967	0.6807	0.6790 / 0.6756	0.6553 / 0.6871